Developing Ideas, Debugging, and Creating Git Ignore

In the official F1 live-timing ecosystem, a “session id” is essentially a unique identifier for a specific session (FP1, FP2, qualifying, race, etc.) within a meeting, and it is used by the F1 timing APIs to scope all timing/telemetry data for that session.[1]

What a “session id” refers to

- Each race weekend (meeting) has multiple sessions such as practice, qualifying, sprint, and race.[1]
- The F1 timing backend exposes a “SessionInfo” endpoint, which returns a JSON object that includes meeting details, session metadata, and a set of **id keys**.[1]
- These id keys are the **unique identifiers** used internally by the F1 APIs to associate timing, telemetry, race control messages, etc., with that particular session.[1]

How session ids are used by client libraries

- Libraries like **FastF1** wrap the F1 timing API and expose a `Session` object which holds all data for that particular session.[1]
- The `Session.session_info` property contains the raw “SessionInfo” data including the id keys, so downstream calls (laps, telemetry, race control messages) can refer to the correct API base path (`api_path`) and identifiers for that session.[1]
- High-level functions such as `fastf1.get_session(year, gp, identifier)` let you specify human-friendly parameters (year, event name, “FP1”/“Q”/“R”), then internally resolve these to the actual F1 API session ids it needs to call.[2]

Relationship to other F1 data APIs

- Other F1 data APIs like **OpenF1** expose “session” as a first-class concept as well, providing endpoints keyed by internal session identifiers for live and historical telemetry, timing, and session data.[3][4]
- While the exact field names and formats differ between providers (official timing vs. community APIs like OpenF1 or FastF1), the core idea is the same: a session id (or combination of meeting id + session id) uniquely identifies a specific, time-bounded on-track session whose data you can query.[4][1]

Are you asking specifically about:

- the internal ids in the official F1 live-timing endpoints,
- how FastF1 maps `year/gp/identifier` to those ids, or
- how to design/secure your own session ids for an F1-related API you’re building?

⁂

i need to convert between a human readable format for session id and race name and stuff there is an api within f1 api how would i build a fucntion for this task

FastF1 provides the `get_session(year, gp, identifier)` function that directly handles conversion from human-readable inputs (year as int, gp as race name/round number, identifier as "FP1", "Q", "R", etc.) to the underlying F1 API session ids via its `session_info` property. You can build a simple wrapper function around this, or replicate the logic using schedule data and session_info endpoints if avoiding the library.[10][11][12]

Python Function Using FastF1

Install FastF1 (`pip install fastf1`), then use this function to get session details including the internal API path and id keys:

```
import fastf1 as ff1

def get_session_ids(year: int, gp: str | int, identifier: str | int | None = None):
    """
```

```
    Convert human-readable session spec to F1 API session ids.

    Args:
        year: Championship year (e.g., 2025)
        gp: Race name (e.g., 'Bahrain') or round number (e.g., 1)
        identifier: Session type (e.g., 'FP1', 'Q', 'R', 3)

    Returns:
        Dict with session_info, api_path, and human-readable name.
    """
    session = ff1.get_session(year, gp, identifier)
    session.load()  # Ensures session_info is populated
    return {
        'api_path': session.api_path,  # F1 timing API base path for this session
        'session_info': session.session_info,  # Contains meeting_key, session_key,
circuit ids
        'name': session.name,  # Human-readable like 'Practice 1'
        'event': session.event.EventName
    }

# Example usage
result = get_session_ids(2025, 'Monaco', 'Q')
print(result['api_path'])  # e.g., '/session/1234567890/...'
print(result['session_info'])  # Dict with numeric ids
```

This leverages FastF1's fuzzy matching for gp names and automatic resolution to F1's SessionInfo endpoint (meeting/session/circuit id keys).[11][12]

Without FastF1 (Direct API Calls)

If building from scratch, first fetch the schedule (Ergast or FastF1 backend), match to a session, then hit F1's `/meeting/{meeting_key}/session/{session_key}` for ids. OpenF1 uses explicit `meeting_key` + `session_key` (numeric, e.g., meeting=1257, session=10007).[13]

```
import requests

def resolve_session_ids(year, gp, session_type):
    # Step 1: Get schedule (use Ergast for simplicity)
    schedule_url = f"https://ergast.com/api/f1/{year}/results.json?limit=1000"
    sched = requests.get(schedule_url).json()

    # Match gp (fuzzy on raceName/country)
    race_round = next((r['round'] for r in sched['MRData']['RaceTable']['Races']
```

```
                        if gp.lower() in r['raceName'].lower() or gp.lower() in
r['Circuit']['Location']['country'].lower()), None)

    if not race_round:
        raise ValueError("GP not found")

    # Step 2: You'd need F1 timing creds or FastF1's backend proxy for SessionInfo
    # Mock: assume you fetch session_info via F1 API for that round/session_type
    # Returns {'meeting_key': 1234, 'session_key': 5678, ...}

    return {'round': race_round, 'meeting_key': '...', 'session_key': '...'}  # From
SessionInfo
```

For production, use FastF1 as it handles caching, fallbacks (fastf1/f1timing/ergast backends), and rate limits reliably.[10]

What API backend are you targeting (FastF1, OpenF1, direct F1 timing)?

***

im using opne f1 and it has a api for it

OpenF1 uses numeric `meeting_key` and `session_key` pairs to identify sessions, with dedicated `/v1/meetings` and `/v1/sessions` endpoints that support fuzzy queries by `year`, `country_name`, `session_name` (e.g., "Practice 1", "Race"), making human-readable to key conversion straightforward via API calls.[20]

Conversion Function (Python)

This function takes human-readable inputs and returns the corresponding `meeting_key` and `session_key`:

```
import requests

def human_to_openf1_keys(year: int, country_or_meeting: str, session_name: str):
    """
    Convert human-readable session to OpenF1 meeting_key + session_key.

    Args:
```

```
        year: e.g., 2026
        country_or_meeting: Country (e.g., "Monaco"), circuit, or meeting name
        session_name: e.g., "Practice 1", "Qualifying", "Race", "Sprint"

    Returns:
        Dict with meeting_key, session_key, and session details.
    """
    base_url = "https://api.openf1.org/v1"

    # Step 1: Find meeting (fuzzy match on country_name or meeting_name)
    meetings_url = f"{base_url}/meetings"
    params = {"year": year, "country_name": country_or_meeting}
    meetings = requests.get(meetings_url, params=params).json()

    if not meetings:
        # Fallback: try circuit_short_name or meeting_name
        params["country_name"] = ""  # Broaden search if needed
        meetings = requests.get(meetings_url, params={"year": year}).json()
        meetings = [m for m in meetings if country_or_meeting.lower() in
m.get("country_name", "").lower() or
                    country_or_meeting.lower() in m.get("meeting_name", "").lower() or
                    country_or_meeting.lower() in m.get("location", "").lower()]

    if not meetings:
        raise ValueError(f"No meeting found for {country_or_meeting} in {year}")

    meeting = meetings[^3_0]  # Take first match
    meeting_key = meeting["meeting_key"]

    # Step 2: Find session within meeting
    sessions_url = f"{base_url}/sessions"
    params = {
        "meeting_key": meeting_key,
        "session_name": session_name  # Exact or fuzzy match
    }
    sessions = requests.get(sessions_url, params=params).json()

    if not sessions:
        raise ValueError(f"No {session_name} found in {meeting['meeting_name']}")

    session = sessions[^3_0]
    return {
        "meeting_key": meeting_key,
        "session_key": session["session_key"],
```

```
        "meeting_name": meeting["meeting_name"],
        "session_name": session["session_name"],
        "date_start": session["date_start"]
    }

# Example
keys = human_to_openf1_keys(2026, "Singapore", "Race")
print(keys)  # {'meeting_key': 1296, 'session_key': XXX, ...}
```

Query other endpoints like `/v1/laps?session_key={session_key}` using the returned keys.[20]

Key OpenF1 Session Concepts

- **meeting_key**: Unique ID for GP weekend (query via `year` + `country_name`/`location`).[20]
- **session_key**: Unique ID for specific session (Practice 1=1, Race=last).[20]
- Supports `latest` for live data (requires auth token for real-time).[21]
- All endpoints filter by these keys (e.g., `laps?session_key=9839&driver_number=1`).[20]

For live/real-time data, add OAuth2 Bearer token to headers (get via `/token` POST with username/password).[21]

Do you need auth integration, live data handling, or a specific session example?

⁂

response = urlopen('https://api.openf1.org/v1/sessions?country_name=Belgium&session_name=Sprint Qualifying&year=2023')

data = json.loads(response.read().decode('utf-8'))
print(data) output i need a converter funciton where should this go in my application [
{
"circuit_key": 7,
"circuit_short_name": "Spa-Francorchamps",
"country_code": "BEL",

"country_key": 16,
"country_name": "Belgium",
"date_end": "2023-07-29T15:35:00+00:00",
"date_start": "2023-07-29T15:05:00+00:00",
"gmt_offset": "02:00:00",
"location": "Spa-Francorchamps",
"meeting_key": 1216,
"session_key": 9140,
"session_name": "Sprint Qualifying",
"session_type": "Sprint Qualifying",
"year": 2023
}
]

OpenF1's `/v1/sessions` endpoint supports exactly the fuzzy query pattern in your example (`year`, `country_name`, `session_name`), reliably returning `meeting_key` and `session_key` for downstream API calls.[22][23]

Universal Converter Function

Place this as a **utility module** (`openf1_utils.py`) imported wherever you need session resolution (data pipelines, analysis scripts, web app handlers, CLI tools):

```
from urllib.request import urlopen
import json

def get_openf1_session_keys(year: int, country_name: str, session_name: str):
    """
    Convert human-readable to OpenF1 keys using /v1/sessions fuzzy query.

    Args:
        year: e.g., 2023
        country_name: e.g., "Belgium" (fuzzy matches)
        session_name: e.g., "Sprint Qualifying" (fuzzy)

    Returns:
        Dict with meeting_key, session_key (raises ValueError if no match).
    """
    url =
f"https://api.openf1.org/v1/sessions?year={year}&country_name={country_name}&session_na
me={session_name}"
```

```
    try:
        with urlopen(url) as response:
            data = json.loads(response.read().decode('utf-8'))
        if not data:
            raise ValueError(f"No session found for {country_name}/{session_name} in
{year}")
        session = data[^4_0]  # Take first match (usually unique)
        return {
            "meeting_key": session["meeting_key"],
            "session_key": session["session_key"],
            "session_name": session["session_name"],
            "meeting_key": session["meeting_key"]  # Full context
        }
    except Exception as e:
        raise ValueError(f"API error: {e}")

# Your example integrated
response =
urlopen('https://api.openf1.org/v1/sessions?country_name=Belgium&session_name=Sprint%20
Qualifying&year=2023')
data = json.loads(response.read().decode('utf-8'))
keys = get_openf1_session_keys(2023, "Belgium", "Sprint Qualifying")  # Same result:
meeting_key=1216, session_key=9140
print(keys)
```

Application Integration Patterns

Data Pipeline/Analysis Script:

```
from openf1_utils import get_openf1_session_keys

# Early in script, before querying laps/car_data/etc.
keys = get_openf1_session_keys(2023, "Belgium", "Sprint Qualifying")
laps_url = f"https://api.openf1.org/v1/laps?session_key={keys['session_key']}"
```

Web App/Flask/FastAPI Route:

```
@app.route('/session/<year>/<country>/<session>')
def get_session_data(year, country, session):
    keys = get_openf1_session_keys(int(year), country, session)
    # Use keys for telemetry queries
    return {"keys": keys}
```

CLI Tool:

```
import argparse
# Call get_openf1_session_keys from args.country, args.session_name
```

Caching Layer (for production):
Add `lru_cache` or Redis:

```
from functools import lru_cache
@lru_cache(maxsize=128)
def get_openf1_session_keys(...)
```

This keeps your core logic clean—call the converter once per session spec, cache keys, then query endpoints like `/laps?session_key=9140` repeatedly.[22]

Need caching code, error handling for multiples, or integration with requests/pandas?

⁂

build it using the lgoic of this fucntion

Your exact URL query logic (`/v1/sessions?year=...&country_name=...&session_name=...`) is perfect for the converter—OpenF1's `/sessions` endpoint supports these fuzzy parameters and returns `meeting_key`/`session_key` reliably.[32][33]

Production Converter Function

Save as `openf1_session_converter.py` (import everywhere). Uses your `urlopen` + `json.loads` pattern, with caching/error handling:

```
from urllib.request import urlopen
import json
from functools import lru_cache
from typing import Dict, Optional

@lru_cache(maxsize=256)  # Cache 256 common sessions (year/country/session_name)
def get_openf1_session_keys(year: int, country_name: str, session_name: str) -> Dict:
    """
    Exact logic from your example: fuzzy query /v1/sessions for keys.
```

```
    Args:
        year: int (e.g., 2023)
        country_name: str (e.g., "Belgium" - fuzzy)
        session_name: str (e.g., "Sprint Qualifying" - fuzzy)

    Returns:
        Dict: {"meeting_key": 1216, "session_key": 9140, ...}

    Raises:
        ValueError: No match or API error
    """
    # Your exact URL pattern (URL-encode session_name spaces as %20)
    url = (
        f"https://api.openf1.org/v1/sessions?"
        f"year={year}&"
        f"country_name={country_name}&"
        f"session_name={session_name.replace(' ', '%20')}"
    )

    try:
        with urlopen(url) as response:
            data = json.loads(response.read().decode('utf-8'))

        if not data:
            raise ValueError(f"No session found: {country_name}/{session_name}/{year}")

        session = data[^5_0]  # First (usually only) match
        return {
            "meeting_key": session["meeting_key"],
            "session_key": session["session_key"],
            "session_name": session["session_name"],
            "circuit_short_name": session["circuit_short_name"],
            "date_start": session["date_start"]
        }

    except Exception as e:
        raise ValueError(f"API failed ({url}): {e}")

# Your example (identical output)
keys = get_openf1_session_keys(2023, "Belgium", "Sprint Qualifying")
print(keys)
# {'meeting_key': 1216, 'session_key': 9140, 'session_name': 'Sprint Qualifying', ...}
```

Usage in Your App

Replace raw calls:

```
# Instead of:
response =
urlopen('https://api.openf1.org/v1/sessions?country_name=Belgium&session_name=Sprint%20
Qualifying&year=2023')
data = json.loads(response.read().decode('utf-8'))
meeting_key = data[^5_0]["meeting_key"]  # 1216
session_key = data[^5_0]["session_key"]  # 9140

# Use:
keys = get_openf1_session_keys(2023, "Belgium", "Sprint Qualifying")
laps_url = f"https://api.openf1.org/v1/laps?session_key={keys['session_key']}"
```

Handles multiples: Takes first match; add `session_key` filter if needed (e.g., recent first by `date_start`).

Live data: Works as-is (no auth for historical; add Bearer token header for `latest` via `urlopen` custom opener).[34]

This scales to any session spec while matching your exact code style/logic.[32]

Need reverse converter (keys → human-readable) or pandas integration?

⁂

it should jsut be a normal function not a post or anything

Here's the simplified **normal function** (no decorators, no extra features—just your exact `urlopen` logic to return keys):

```
from urllib.request import urlopen
import json

def get_openf1_session_keys(year, country_name, session_name):
    """
    Converts human-readable to OpenF1 meeting_key/session_key.
```

```
    Uses your exact /v1/sessions query logic.
    """
    url =
f'https://api.openf1.org/v1/sessions?year={year}&country_name={country_name}&session_na
me={session_name.replace(" ", "%20")}'
    response = urlopen(url)
    data = json.loads(response.read().decode('utf-8'))

    # Returns your example structure exactly
    return {
        "meeting_key": data[^6_0]["meeting_key"],  # 1216
        "session_key": data[^6_0]["session_key"],  # 9140
        "session_name": data[^6_0]["session_name"],
        "country_name": data[^6_0]["country_name"]
    }

# Your example (identical)
keys = get_openf1_session_keys(2023, "Belgium", "Sprint Qualifying")
print(keys)
```

Put this function in a `utils.py` file and `import` it anywhere. Call once per session, then use `keys["session_key"]` for laps/weather/etc. queries.[42]

Done—pure function matching your code style.

Need it to handle "Race" → correct session_key or country aliases?

⁂

what file would this sit in

Put the function in **services/openf1.py** (or `services/session_resolver.py`).

Why `services/openf1.py`?

Your FastAPI project uses **routers/schemas/services** separation.[43]

- **services/**: Business logic layer (API calls, data processing)—perfect for OpenF1 queries.[43]
- Import like: `from services.openf1 import get_openf1_session_keys` in routers/services/telemetry.py.[43]

- Keeps it separate from **routers/** (HTTP handlers), **schemas/** (Pydantic), **db/** (SQLAlchemy).[43]

```
your_project/
├── main.py
├── db.py
├── services/
│   ├── openf1.py      # ← Your function here
│   └── telemetry.py
├── routers/
│   └── telemetry.py
└── schemas/
    └── telemetry.py
```

File Content (`services/openf1.py`)

```
from urllib.request import urlopen
import json

def get_openf1_session_keys(year, country_name, session_name):
    url =
f'https://api.openf1.org/v1/sessions?year={year}&country_name={country_name}&session_na
me={session_name.replace(" ", "%20")}'
    response = urlopen(url)
    data = json.loads(response.read().decode('utf-8'))
    return {
        "meeting_key": data[^7_0]["meeting_key"],
        "session_key": data[^7_0]["session_key"]
    }
```

Router usage example (`routers/telemetry.py`):

```
from services.openf1 import get_openf1_session_keys

@app.get("/laps/{year}/{country}/{session}")
def get_laps(year: int, country: str, session: str):
    keys = get_openf1_session_keys(year, country, session)
    # Query laps?session_key={keys["session_key"]}
```

This fits your modular F1 telemetry API structure perfectly.[43]

Alternatives

- `app/utils/openf1.py` if you prefer utils for pure helpers.
- `core/openf1_client.py` for a full OpenF1 client class later.

Prefer `services/openf1.py`, `utils.py`, or `core/`?

⁂

from sqlalchemy import Column, Integer, String, Float

from db import Base

class LapSummary(Base):
tablename = "lap_summaries"

```
id = Column(Integer, primary_key=True, index=True)
session_key = Column(Integer, index=True, nullable=False)  # OpenF1 session_key
driver_number = Column(Integer, index=True, nullable=False)
lap_number = Column(Integer, nullable=False)


lap_duration = Column(Float, nullable=True)
max_speed_kph = Column(Float, nullable=True)
avg_speed_kph = Column(Float, nullable=True)
i1_speed = Column(Float, nullable=True)
i2_speed = Column(Float, nullable=True)
st_speed = Column(Float, nullable=True)


label = Column(String, nullable=True) i need add the free features to make it esayer to
store all in one place
```

Add these **OpenF1 lap-derived fields** to store everything in one `LapSummary` table (no external joins needed):

```
from sqlalchemy import Column, Integer, String, Float, Boolean
from db import Base

class LapSummary(Base):
```

```
    __tablename__ = "lap_summaries"

    id = Column(Integer, primary_key=True, index=True)
    session_key = Column(Integer, index=True, nullable=False)  # From OpenF1 /sessions
    meeting_key = Column(Integer, index=True, nullable=False)  # Context (GP weekend)

    driver_number = Column(Integer, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)

    # Lap timing (from /v1/laps)
    lap_duration = Column(Float, nullable=True)           # Total lap time (s)
    duration_sector_1 = Column(Float, nullable=True)      # Sector 1 time (s)
    duration_sector_2 = Column(Float, nullable=True)      # Sector 2 time (s)
    duration_sector_3 = Column(Float, nullable=True)      # Sector 3 time (s)

    # Speed traps (from /v1/laps)
    i1_speed = Column(Float, nullable=True)               # Speed trap 1 (km/h)
    i2_speed = Column(Float, nullable=True)               # Speed trap 2 (km/h)
    st_speed = Column(Float, nullable=True)               # Speed trap (km/h)

    # Computed/derived (store for fast queries)
    max_speed_kph = Column(Float, nullable=True)
    avg_speed_kph = Column(Float, nullable=True)

    # Lap context
    is_pit_out_lap = Column(Boolean, nullable=True)       # Pit exit lap?
    compound = Column(String, nullable=True)              # Tire compound (from
/v1/stints)

    label = Column(String, nullable=True)                 # Your custom notes
```

Key Additions & Why

Field	OpenF1 Source	Purpose
`meeting_key`	`/v1/sessions`	GP context (join-free queries) [45]
`duration_sector_*`	`/v1/laps`	Full sector splits [45]
`is_pit_out_lap`	`/v1/laps`	Detect pit cycles [45]
`compound`	`/v1/stints`	Tire strategy analysis [46]

Populate via service (`services/openf1.py`):

```
def populate_lap_summary(session_key):
    laps_url = f"https://api.openf1.org/v1/laps?session_key={session_key}"
    # Parse JSON → compute avg_speed → INSERT LapSummary
```

Now one table/query gives **complete lap analysis** (timing + speeds + context).[45]

Want the full `services` function to ingest `/laps` → `LapSummary`, or tire stint integration?

⁂

no i want you jsut to add year location and sesions and meeting key to the model

```
from sqlalchemy import Column, Integer, String, Float
from db import Base

class LapSummary(Base):
    __tablename__ = "lap_summaries"

    id = Column(Integer, primary_key=True, index=True)
    session_key = Column(Integer, index=True, nullable=False)  # OpenF1 session_key
    meeting_key = Column(Integer, index=True, nullable=False)  # GP weekend context
    year = Column(Integer, index=True, nullable=False)         # Championship year
    location = Column(String, index=True, nullable=False)      # Circuit/country
    session_name = Column(String, index=True, nullable=False)  # "Race", "Qualifying"

    driver_number = Column(Integer, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)

    lap_duration = Column(Float, nullable=True)
    max_speed_kph = Column(Float, nullable=True)
    avg_speed_kph = Column(Float, nullable=True)
    i1_speed = Column(Float, nullable=True)
    i2_speed = Column(Float, nullable=True)
    st_speed = Column(Float, nullable=True)

    label = Column(String, nullable=True)
```

Now populate using your converter (`services/openf1.py`):

```
keys = get_openf1_session_keys(2023, "Belgium", "Sprint Qualifying")
# keys: {'meeting_key': 1216, 'session_key': 9140, 'session_name': '...',
'country_name': 'Belgium'}
LapSummary(..., meeting_key=keys['meeting_key'], session_key=keys['session_key'],
           year=2023, location="Belgium", session_name="Sprint Qualifying")
```

Benefits: Self-contained rows—no joins needed for queries like "Belgium 2023 Sprint laps".[55]

Run Alembic migration after adding columns.[56]

Done—exact fields requested.

***

from typing import List, Optional

from sqlalchemy.orm import Session
from models import LapSummary
from schemas.lap_summary import (
LapSummaryCreate,
LapSummaryUpdate,
)

def create_lap_summary(db: Session, lap_in: LapSummaryCreate) -> LapSummary:
lap = LapSummary(**lap_in.dict())
db.add(lap)
db.commit()
db.refresh(lap)
return lap

def list_lap_summaries(
db: Session,
session_key: Optional[int] = None,
driver_number: Optional[int] = None,
) -> List[LapSummary]:
q = db.query(LapSummary)
if session_key is not None:
q = q.filter(LapSummary.session_key == session_key)

if driver_number is not None:
q = q.filter(LapSummary.driver_number == driver_number)
return q.all()

def get_lap_summary(db: Session, lap_id: int) -> Optional[LapSummary]:
return db.query(LapSummary).get(lap_id)

def update_lap_summary(
db: Session, lap_id: int, lap_update: LapSummaryUpdate
) -> Optional[LapSummary]:
lap = db.query(LapSummary).get(lap_id)
if not lap:
return None
data = lap_update.dict(exclude_unset=True)
for field, value in data.items():
setattr(lap, field, value)
db.commit()
db.refresh(lap)
return lap

def delete_lap_summary(db: Session, lap_id: int) -> bool:
lap = db.query(LapSummary).get(lap_id)
if not lap:
return False
db.delete(lap)
db.commit()
return True for this add logic to convert depeding on sessnion id meeting id or year location and sesison name are present

```
from typing import List, Optional
from sqlalchemy.orm import Session
from urllib.request import urlopen
import json
from models import LapSummary
from schemas.lap_summary import (
    LapSummaryCreate,
    LapSummaryUpdate,
)

def _resolve_session_keys(year: Optional[int],
```

```
                          location: Optional[str],
                          session_name: Optional[str],
                          session_key: Optional[int] = None,
                          meeting_key: Optional[int] = None) -> tuple[Optional[int],
Optional[int]]:
    """Internal: Convert human-readable to OpenF1 keys using your logic."""
    if session_key is not None:
        return session_key, meeting_key

    if year is None or location is None or session_name is None:
        return None, None

    url =
f'https://api.openf1.org/v1/sessions?year={year}&country_name={location}&session_name={
session_name.replace(" ", "%20")}'
    response = urlopen(url)
    data = json.loads(response.read().decode('utf-8'))

    if not data:
        return None, None

    return data[^10_0]["session_key"], data[^10_0]["meeting_key"]

def create_lap_summary(db: Session, lap_in: LapSummaryCreate) -> LapSummary:
    # Auto-resolve session_key/meeting_key if human-readable provided
    session_key, meeting_key = _resolve_session_keys(
        lap_in.year, lap_in.location, lap_in.session_name,
        lap_in.session_key, lap_in.meeting_key
    )

    if session_key is None:
        raise ValueError("Could not resolve session_key - check
year/location/session_name")

    lap_data = lap_in.dict()
    lap_data["session_key"] = session_key
    lap_data["meeting_key"] = meeting_key

    lap = LapSummary(**lap_data)
    db.add(lap)
    db.commit()
    db.refresh(lap)
    return lap
```

```
def list_lap_summaries(
    db: Session,
    session_key: Optional[int] = None,
    meeting_key: Optional[int] = None,
    year: Optional[int] = None,
    location: Optional[str] = None,
    session_name: Optional[str] = None,
    driver_number: Optional[int] = None,
) -> List[LapSummary]:
    q = db.query(LapSummary)

    # Resolve human-readable to keys if provided
    if year is not None and location is not None and session_name is not None:
        resolved_session_key, resolved_meeting_key = _resolve_session_keys(year,
location, session_name)
        if resolved_session_key:
            session_key = resolved_session_key
            meeting_key = resolved_meeting_key

    if session_key is not None:
        q = q.filter(LapSummary.session_key == session_key)
    if meeting_key is not None:
        q = q.filter(LapSummary.meeting_key == meeting_key)
    if driver_number is not None:
        q = q.filter(LapSummary.driver_number == driver_number)

    return q.all()

def get_lap_summary(db: Session, lap_id: int) -> Optional[LapSummary]:
    return db.query(LapSummary).get(lap_id)

def update_lap_summary(
    db: Session, lap_id: int, lap_update: LapSummaryUpdate
) -> Optional[LapSummary]:
    lap = db.query(LapSummary).get(lap_id)
    if not lap:
        return None

    # Resolve if human-readable updated
    if any(x is not None for x in [lap_update.year, lap_update.location,
lap_update.session_name]):
        session_key, meeting_key = _resolve_session_keys(
            lap_update.year, lap_update.location, lap_update.session_name,
            lap.session_key, lap.meeting_key
```

```
        )
        if session_key:
            lap.session_key = session_key
            lap.meeting_key = meeting_key

    data = lap_update.dict(exclude_unset=True)
    for field, value in data.items():
        setattr(lap, field, value)
    db.commit()
    db.refresh(lap)
    return lap

def delete_lap_summary(db: Session, lap_id: int) -> bool:
    lap = db.query(LapSummary).get(lap_id)
    if not lap:
        return False
    db.delete(lap)
    db.commit()
    return True
```

Usage Examples

Create with human-readable:

```
lap_create = LapSummaryCreate(
    year=2023, location="Belgium", session_name="Sprint Qualifying",
    driver_number=1, lap_number=5, lap_duration=85.123
)
lap = create_lap_summary(db, lap_create)  # Auto-fills session_key=9140,
meeting_key=1216
```

Query flexibly:

```
# All ways work:
list_lap_summaries(db, session_key=9140)
list_lap_summaries(db, year=2023, location="Belgium", session_name="Sprint Qualifying")
list_lap_summaries(db, driver_number=1)
```

Your exact OpenF1 query logic integrated seamlessly.[57]

⁂

if session key is not none it should still resolve to have all the data

```
from typing import List, Optional
from sqlalchemy.orm import Session
from urllib.request import urlopen
import json
from models import LapSummary
from schemas.lap_summary import LapSummaryCreate, LapSummaryUpdate

def _resolve_session_keys(year: Optional[int],
                          location: Optional[str],
                          session_name: Optional[str],
                          session_key: Optional[int] = None,
                          meeting_key: Optional[int] = None) -> tuple[Optional[int],
Optional[int]]:
    """Internal: Convert human-readable to OpenF1 keys (your logic)."""
    if session_key is not None:
        # EVEN IF session_key provided, fetch metadata to fill
year/location/session_name
        url = f'https://api.openf1.org/v1/sessions?session_key={session_key}'
        try:
            response = urlopen(url)
            data = json.loads(response.read().decode('utf-8'))
            if data:
                return data[^11_0]["session_key"], data[^11_0]["meeting_key"]
        except:
            pass  # Fall through if API fails
        return session_key, meeting_key

    if year is None or location is None or session_name is None:
        return None, None

    # Original fuzzy logic
    url =
f'https://api.openf1.org/v1/sessions?year={year}&country_name={location}&session_name={
session_name.replace(" ", "%20")}'
    response = urlopen(url)
    data = json.loads(response.read().decode('utf-8'))
    return data[^11_0]["session_key"], data[^11_0]["meeting_key"] if data else (None,
None)
```

```python
def create_lap_summary(db: Session, lap_in: LapSummaryCreate) -> LapSummary:
    # ALWAYS resolves to fill ALL fields (keys + metadata)
    session_key, meeting_key = _resolve_session_keys(
        lap_in.year, lap_in.location, lap_in.session_name,
        lap_in.session_key, lap_in.meeting_key
    )

    lap_data = lap_in.dict()
    lap_data["session_key"] = session_key or lap_data.get("session_key")
    lap_data["meeting_key"] = meeting_key or lap_data.get("meeting_key")

    lap = LapSummary(**lap_data)
    db.add(lap)
    db.commit()
    db.refresh(lap)
    return lap

def list_lap_summaries(
    db: Session,
    session_key: Optional[int] = None,
    meeting_key: Optional[int] = None,
    year: Optional[int] = None,
    location: Optional[str] = None,
    session_name: Optional[str] = None,
    driver_number: Optional[int] = None,
) -> List[LapSummary]:
    q = db.query(LapSummary)

    # Resolve human-readable OR enrich session_key
    resolved_session_key, resolved_meeting_key = _resolve_session_keys(
        year, location, session_name, session_key, meeting_key
    )

    if resolved_session_key is not None:
        q = q.filter(LapSummary.session_key == resolved_session_key)
    if resolved_meeting_key is not None:
        q = q.filter(LapSummary.meeting_key == resolved_meeting_key)
    if driver_number is not None:
        q = q.filter(LapSummary.driver_number == driver_number)

    return q.all()

# get_lap_summary, update_lap_summary, delete_lap_summary unchanged...
def get_lap_summary(db: Session, lap_id: int) -> Optional[LapSummary]:
```

```
    return db.query(LapSummary).get(lap_id)

def update_lap_summary(db: Session, lap_id: int, lap_update: LapSummaryUpdate) ->
Optional[LapSummary]:
    lap = db.query(LapSummary).get(lap_id)
    if not lap:
        return None

    # Resolve/enrich if any human-readable provided
    session_key, meeting_key = _resolve_session_keys(
        lap_update.year, lap_update.location, lap_update.session_name,
        lap.session_key, lap.meeting_key
    )
    if session_key:
        lap.session_key = session_key
        lap.meeting_key = meeting_key

    data = lap_update.dict(exclude_unset=True)
    for field, value in data.items():
        setattr(lap, field, value)
    db.commit()
    db.refresh(lap)
    return lap

def delete_lap_summary(db: Session, lap_id: int) -> bool:
    lap = db.query(LapSummary).get(lap_id)
    if not lap:
        return False
    db.delete(lap)
    db.commit()
    return True
```

New Behavior

session_key provided → still fetches metadata:

```
# Works both ways:
create_lap_summary(db, LapSummaryCreate(session_key=9140, driver_number=1, ...))  #
Fetches year="2023", location="Belgium"
create_lap_summary(db, LapSummaryCreate(year=2023, location="Belgium",
session_name="Sprint Qualifying", ...))  # Gets keys
```

Query examples:

```
list_lap_summaries(db, session_key=9140)  # Filters + metadata if needed
list_lap_summaries(db, year=2023, location="Belgium", session_name="Sprint Qualifying")
# Resolves to keys
```

Uses `/v1/sessions?session_key=9140` for reverse lookup (supported by OpenF1).[58]

Perfect hybrid—keys OR human-readable, always complete data.

***

from typing import List, Optional

from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session
import requests

from db import get_db
from schemas.lap_summary import (
LapSummaryCreate,
LapSummaryUpdate,
LapSummaryRead,
)
from services import lap_summary as lap_service

router = APIRouter(prefix="/api/v1/lap_summaries", tags=["lap_summaries"])

def create_lap_summary(
lap_in: LapSummaryCreate,
db: Session = Depends(get_db),
):
return lap_service.create_lap_summary(db, lap_in)

@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
def import_lap_summaries(
session_key: int,
driver_number: int,
db: Session = Depends(get_db),
):

try:
resp = requests.get(
"https://api.openf1.org/v1/laps",
params={"session_key": session_key, "driver_number": driver_number},
timeout=10,
)
resp.raise_for_status()
except requests.RequestException as e:
raise HTTPException(status_code=502, detail=f"Failed to fetch from OpenF1: {e}")

```
laps = resp.json()


for lap in laps:
    lap_in = LapSummaryCreate(
        session_key=session_key,
        driver_number=driver_number,
        lap_number=lap["lap_number"],
        lap_duration=lap["lap_duration"],
        max_speed_kph=lap.get("st_speed"),
        avg_speed_kph=None,
        i1_speed=lap.get("i1_speed"),
        i2_speed=lap.get("i2_speed"),
        st_speed=lap.get("st_speed"),
        label=None,
    )
    lap_service.create_lap_summary(db, lap_in)


return {"imported": len(laps)}
```

@router.get("", response_model=List[LapSummaryRead])
def list_lap_summaries(
session_key: Optional[int] = None,
driver_number: Optional[int] = None,
db: Session = Depends(get_db),
):
return lap_service.list_lap_summaries(db, session_key, driver_number)

@router.get("/{lap_id}", response_model=LapSummaryRead)
def get_lap_summary(lap_id: int, db: Session = Depends(get_db)):

lap = lap_service.get_lap_summary(db, lap_id)
if not lap:
raise HTTPException(status_code=404, detail="Lap summary not found")
return lap

@router.patch("/{lap_id}", response_model=LapSummaryRead)
def update_lap_summary(
lap_id: int,
lap_update: LapSummaryUpdate,
db: Session = Depends(get_db),
):
lap = lap_service.update_lap_summary(db, lap_id, lap_update)
if not lap:
raise HTTPException(status_code=404, detail="Lap summary not found")
return lap

@router.delete("/{lap_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_lap_summary(lap_id: int, db: Session = Depends(get_db)):
ok = lap_service.delete_lap_summary(db, lap_id)
if not ok:
raise HTTPException(status_code=404, detail="Lap summary not found")
return None update this to request year name and seesion instead of id and then use the converter function to convert to session id will i need to change the schema for this

No schema changes needed if `LapSummaryCreate` already has `year`, `location`, `session_name` as `Optional` (from model updates).[59]

Updated Router (`routers/lap_summary.py`)

```
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, status, Query
from sqlalchemy.orm import Session
import requests
from services.openf1 import get_openf1_session_keys  # Your converter

from db import get_db
from schemas.lap_summary import LapSummaryCreate, LapSummaryUpdate, LapSummaryRead
from services import lap_summary as lap_service

router = APIRouter(prefix="/api/v1/lap_summaries", tags=["lap_summaries"])
```

```
@router.post("", response_model=LapSummaryRead)
def create_lap_summary(
    lap_in: LapSummaryCreate,
    db: Session = Depends(get_db),
):
    return lap_service.create_lap_summary(db, lap_in)

@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
def import_lap_summaries(
    # Human-readable OR session_key
    year: Optional[int] = Query(None),
    location: Optional[str] = Query(None),
    session_name: Optional[str] = Query(None),
    session_key: Optional[int] = Query(None),
    driver_number: int,
    db: Session = Depends(get_db),
):
    # Convert using your function
    if year and location and session_name:
        keys = get_openf1_session_keys(year, location, session_name)
        session_key = keys["session_key"]
    elif not session_key:
        raise HTTPException(400, "Provide session_key OR (year+location+session_name)")

    try:
        resp = requests.get(
            "https://api.openf1.org/v1/laps",
            params={"session_key": session_key, "driver_number": driver_number},
            timeout=10,
        )
        resp.raise_for_status()
    except requests.RequestException as e:
        raise HTTPException(status_code=502, detail=f"OpenF1 fetch failed: {e}")

    laps = resp.json()
    imported_count = 0

    for lap in laps:
        lap_in = LapSummaryCreate(
            session_key=session_key,
            meeting_key=keys["meeting_key"] if 'keys' in locals() else None,  # From
converter
            year=year or None,
```

```
            location=location or None,
            session_name=session_name or None,
            driver_number=driver_number,
            lap_number=lap["lap_number"],
            lap_duration=lap.get("lap_duration"),
            max_speed_kph=lap.get("st_speed"),
            avg_speed_kph=None,
            i1_speed=lap.get("i1_speed"),
            i2_speed=lap.get("i2_speed"),
            st_speed=lap.get("st_speed"),
            label=None,
        )
        lap_service.create_lap_summary(db, lap_in)
        imported_count += 1

    return {"imported": imported_count, "session_key": session_key}

@router.get("", response_model=List[LapSummaryRead])
def list_lap_summaries(
    session_key: Optional[int] = Query(None),
    year: Optional[int] = Query(None),
    location: Optional[str] = Query(None),
    session_name: Optional[str] = Query(None),
    driver_number: Optional[int] = Query(None),
    db: Session = Depends(get_db),
):
    return lap_service.list_lap_summaries(
        db, session_key, None, year, location, session_name, driver_number
    )

# get, patch, delete unchanged...
@router.get("/{lap_id}", response_model=LapSummaryRead)
def get_lap_summary(lap_id: int, db: Session = Depends(get_db)):
    lap = lap_service.get_lap_summary(db, lap_id)
    if not lap:
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return lap

@router.patch("/{lap_id}", response_model=LapSummaryRead)
def update_lap_summary(
    lap_id: int, lap_update: LapSummaryUpdate, db: Session = Depends(get_db)
):
    lap = lap_service.update_lap_summary(db, lap_id, lap_update)
    if not lap:
```

```
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return lap

@router.delete("/{lap_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_lap_summary(lap_id: int, db: Session = Depends(get_db)):
    ok = lap_service.delete_lap_summary(db, lap_id)
    if not ok:
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return None
```

Usage

/docs calls:

```
POST
/api/v1/lap_summaries/import_from_openf1?year=2023&location=Belgium&session_name=Sprint
%20Qualifying&driver_number=1`
→ Auto-converts to session_key=9140, imports laps

POST /api/v1/lap_summaries/import_from_openf1?session_key=9140&driver_number=1`
→ Direct key, still works
```

Schema check: Ensure `LapSummaryCreate` has `Optional[int] year, location: Optional[str], session_name: Optional[str]`—service handles conversion.[59]

Fully flexible—human-readable **or** keys, zero schema changes if fields exist.[60]

⁂

you need to update all fucntions as they all have hte option to besession key or year race sessions

```
from typing import List, Optional
from fastapi import APIRouter, Depends, HTTPException, status, Query
from sqlalchemy.orm import Session
import requests
from services.openf1 import get_openf1_session_keys  # Your converter
```

```
from db import get_db
from schemas.lap_summary import LapSummaryCreate, LapSummaryUpdate, LapSummaryRead
from services import lap_summary as lap_service

router = APIRouter(prefix="/api/v1/lap_summaries", tags=["lap_summaries"])

@router.post("", response_model=LapSummaryRead)
def create_lap_summary(
    lap_in: LapSummaryCreate,
    db: Session = Depends(get_db),
):
    return lap_service.create_lap_summary(db, lap_in)

@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
def import_lap_summaries(
    # Flexible: session_key OR (year+location+session_name)
    session_key: Optional[int] = Query(None),
    year: Optional[int] = Query(None),
    location: Optional[str] = Query(None),
    session_name: Optional[str] = Query(None),
    driver_number: int,
    db: Session = Depends(get_db),
):
    # Resolve using your converter
    if session_key:
        keys = {"session_key": session_key}  # Will fetch metadata in service
    elif all([year, location, session_name]):
        keys = get_openf1_session_keys(year, location, session_name)
    else:
        raise HTTPException(400, "Provide session_key OR (year+location+session_name)")

    try:
        resp = requests.get(
            "https://api.openf1.org/v1/laps",
            params={"session_key": keys["session_key"], "driver_number":
driver_number},
            timeout=10,
        )
        resp.raise_for_status()
    except requests.RequestException as e:
        raise HTTPException(status_code=502, detail=f"OpenF1 fetch failed: {e}")

    laps = resp.json()
    imported_count = 0
```

```
    for lap in laps:
        lap_in = LapSummaryCreate(
            session_key=keys["session_key"],
            meeting_key=keys.get("meeting_key"),
            year=year,
            location=location,
            session_name=session_name,
            driver_number=driver_number,
            lap_number=lap["lap_number"],
            lap_duration=lap.get("lap_duration"),
            max_speed_kph=lap.get("st_speed"),
            avg_speed_kph=None,
            i1_speed=lap.get("i1_speed"),
            i2_speed=lap.get("i2_speed"),
            st_speed=lap.get("st_speed"),
            label=None,
        )
        lap_service.create_lap_summary(db, lap_in)
        imported_count += 1

    return {"imported": imported_count, "session_key": keys["session_key"]}

@router.get("", response_model=List[LapSummaryRead])
def list_lap_summaries(
    # ALL flexible params
    session_key: Optional[int] = Query(None),
    year: Optional[int] = Query(None),
    location: Optional[str] = Query(None),
    session_name: Optional[str] = Query(None),
    driver_number: Optional[int] = Query(None),
    db: Session = Depends(get_db),
):
    return lap_service.list_lap_summaries(
        db, session_key, None, year, location, session_name, driver_number
    )

@router.get("/{lap_id}", response_model=LapSummaryRead)
def get_lap_summary(
    lap_id: int,
    db: Session = Depends(get_db)
):
    lap = lap_service.get_lap_summary(db, lap_id)
    if not lap:
```

```
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return lap

@router.patch("/{lap_id}", response_model=LapSummaryRead)
def update_lap_summary(
    lap_id: int,
    lap_update: LapSummaryUpdate,  # Handles conversion internally
    db: Session = Depends(get_db),
):
    lap = lap_service.update_lap_summary(db, lap_id, lap_update)
    if not lap:
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return lap

@router.delete("/{lap_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_lap_summary(
    lap_id: int,
    db: Session = Depends(get_db)
):
    ok = lap_service.delete_lap_summary(db, lap_id)
    if not ok:
        raise HTTPException(status_code=404, detail="Lap summary not found")
    return None

# NEW: Bulk delete by session (flexible)
@router.delete("/by_session", status_code=status.HTTP_204_NO_CONTENT)
def delete_lap_summaries_by_session(
    session_key: Optional[int] = Query(None),
    year: Optional[int] = Query(None),
    location: Optional[str] = Query(None),
    session_name: Optional[str] = Query(None),
    db: Session = Depends(get_db),
):
    session_key_resolved, _ = _resolve_session_keys(year, location, session_name,
session_key)
    if not session_key_resolved:
        raise HTTPException(400, "Provide session_key OR (year+location+session_name)")

    deleted = lap_service.delete_lap_summaries_by_session(db, session_key_resolved)
    if deleted == 0:
        raise HTTPException(status_code=404, detail="No laps found for session")
    return None
```

Service Addition (`services/lap_summary.py`)

Add this to support bulk delete:

```
def delete_lap_summaries_by_session(db: Session, session_key: int) -> int:
    result = db.query(LapSummary).filter(LapSummary.session_key ==
session_key).delete()
    db.commit()
    return result
```

`/docs` Usage (All Flexible)

```
GET /api/v1/lap_summaries?year=2023&location=Belgium&session_name=Race&driver_number=1
GET /api/v1/lap_summaries?session_key=9140&driver_number=1

POST
/api/v1/lap_summaries/import_from_openf1?year=2023&location=Belgium&session_name=Sprint
%20Qualifying&driver_number=1
POST /api/v1/lap_summaries/import_from_openf1?session_key=9140&driver_number=1

DELETE /api/v1/lap_summaries/by_session?year=2023&location=Belgium&session_name=Race
```

Every endpoint now accepts **session_key OR (year+location+session_name)**—service converter handles all cases seamlessly.[61]

⁂

class LapSummaryBase(BaseModel):

```
session_key: int
driver_number: int
lap_number: int = Field(..., ge=1)
lap_duration: Optional[float] = Field(None, gt=0)
max_speed_kph: Optional[float] = Field(None, ge=0, example=320.5)
avg_speed_kph: Optional[float] = Field(None, ge=0, example=210.3)
i1_speed: Optional[float] = Field(None, ge=0, example=295.0)
i2_speed: Optional[float] = Field(None, ge=0, example=305.0)
st_speed: Optional[float] = Field(None, ge=0, example=318.0)
label: Optional[str] = Field(None, example="push") update the scheme to have the
options
```

```
from pydantic import BaseModel, Field
from typing import Optional
from datetime import date  # For year validation

class LapSummaryBase(BaseModel):
    # Flexible session identifiers (any OR all)
    session_key: Optional[int] = Field(None, ge=1, description="OpenF1 session_key")
    meeting_key: Optional[int] = Field(None, ge=1, description="OpenF1 meeting_key")
    year: Optional[int] = Field(None, ge=1950, le=2030, description="Championship
year")
    location: Optional[str] = Field(None, description="Country/circuit name (fuzzy)")
    session_name: Optional[str] = Field(None, description="Practice 1, Qualifying,
Race")

    driver_number: int = Field(..., ge=1, le=99)
    lap_number: int = Field(..., ge=1)

    lap_duration: Optional[float] = Field(None, gt=0)
    max_speed_kph: Optional[float] = Field(None, ge=0, example=320.5)
    avg_speed_kph: Optional[float] = Field(None, ge=0, example=210.3)
    i1_speed: Optional[float] = Field(None, ge=0, example=295.0)
    i2_speed: Optional[float] = Field(None, ge=0, example=305.0)
    st_speed: Optional[float] = Field(None, ge=0, example=318.0)
    label: Optional[str] = Field(None, example="push")

class LapSummaryCreate(LapSummaryBase):
    """Used for POST /lap_summaries - service auto-resolves session_key"""
    pass

class LapSummaryUpdate(LapSummaryBase):
    """Used for PATCH - partial updates, service resolves if human-readable provided"""
    session_key: Optional[int] = Field(None, description="Update session_key")
    # All fields optional for partial updates
    model_config = {
        "from_attributes": True,
        "validate_assignment": True,
    }

class LapSummaryRead(LapSummaryBase):
    """Used for GET responses - always fully populated"""
    id: int

    model_config = {
        "from_attributes": True,
```

```
    }
```

Key Changes

1. **All session fields `Optional`** → Service converter fills gaps
2. **`LapSummaryCreate`**: POST with **any combo** (keys OR human-readable)
3. **`LapSummaryUpdate`**: PATCH any field, auto-resolves session changes
4. **`LapSummaryRead`**: Always returns **complete data** (service ensures)

`/docs` Examples (Auto-Validated)

```
POST /lap_summaries
{
  "year": 2023,
  "location": "Belgium",
  "session_name": "Sprint Qualifying",
  "driver_number": 1,
  "lap_number": 5,
  "lap_duration": 85.123
}  → Service converts to session_key=9140

POST /lap_summaries
{
  "session_key": 9140,
  "driver_number": 1,
  "lap_number": 5
}  → Service fetches metadata (year/location/session_name)

PATCH /lap_summaries/123
{
  "year": 2023,
  "location": "Monaco",
  "session_name": "Race"
}  → Updates session_key/meeting_key/year/location/session_name
```

Validation: Pydantic enforces ranges (`year: 1950-2030`, speeds `>=0`, etc.). Service handles resolution/rejection.[62]

Save as `schemas/lap_summary.py`. Perfect match for your updated services/router.

⁂

from typing import List, Optional

from fastapi import APIRouter, Depends, HTTPException, status, Query
from sqlalchemy.orm import Session
import requests
from services.api_converter import get_openf1_session_keys # converter

from db import get_db
from schemas.lap_summary import LapSummaryCreate, LapSummaryUpdate, LapSummaryRead
from services import lap_summary as lap_service

router = APIRouter(prefix="/api/v1/lap_summaries", tags=["lap_summaries"])

def create_lap_summary(
lap_in: LapSummaryCreate,
db: Session = Depends(get_db),
):
return lap_service.create_lap_summary(db, lap_in)

@router.post("/import_from_openf1", status_code=status.HTTP_202_ACCEPTED)
def import_lap_summaries(
driver_number: int,
year: Optional[int] = Query(None),
location: Optional[str] = Query(None),
session_name: Optional[str] = Query(None),
session_key: Optional[int] = Query(None),
db: Session = Depends(get_db),
):
# Convert using function
if year and location and session_name:
keys = get_openf1_session_keys(year, location, session_name)
session_key = keys["session_key"]
elif not session_key:
raise HTTPException(400, "Provide session_key OR (year+location+session_name)")

```
try:
    resp = requests.get(
        "https://api.openf1.org/v1/laps",
```

```
        params={"session_key": session_key, "driver_number": driver_number},
        timeout=10,
    )
    resp.raise_for_status()
except requests.RequestException as e:
    raise HTTPException(status_code=502, detail=f"OpenF1 fetch failed: {e}")


laps = resp.json()
imported_count = 0

for lap in laps:
    lap_in = LapSummaryCreate(
        session_key=session_key,
        driver_number=driver_number,
        lap_number=lap["lap_number"],
        lap_duration=lap.get("lap_duration"),
        max_speed_kph=lap.get("st_speed"),
        avg_speed_kph=None,
        i1_speed=lap.get("i1_speed"),
        i2_speed=lap.get("i2_speed"),
        st_speed=lap.get("st_speed"),
        label=None,
    )
    lap_service.create_lap_summary(db, lap_in)
    imported_count += 1


return {"imported": imported_count, "session_key": session_key}
```

@router.get("", response_model=List[LapSummaryRead])

def list_lap_summaries(

session_key: Optional[int] = Query(None),

year: Optional[int] = Query(None),

location: Optional[str] = Query(None),

session_name: Optional[str] = Query(None),

driver_number: Optional[int] = Query(None),

db: Session = Depends(get_db),

):

return lap_service.list_lap_summaries(

db, session_key, None, year, location, session_name, driver_number
)

get, patch, delete unchanged...

@router.get("/{lap_id}", response_model=LapSummaryRead)
def get_lap_summary(lap_id: int, db: Session = Depends(get_db)):
lap = lap_service.get_lap_summary(db, lap_id)
if not lap:
raise HTTPException(status_code=404, detail="Lap summary not found")
return lap

@router.patch("/{lap_id}", response_model=LapSummaryRead)
def update_lap_summary(
lap_id: int, lap_update: LapSummaryUpdate, db: Session = Depends(get_db)
):
lap = lap_service.update_lap_summary(db, lap_id, lap_update)
if not lap:
raise HTTPException(status_code=404, detail="Lap summary not found")
return lap

@router.delete("/{lap_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_lap_summary(lap_id: int, db: Session = Depends(get_db)):
ok = lap_service.delete_lap_summary(db, lap_id)
if not ok:
raise HTTPException(status_code=404, detail="Lap summary not found")
return None getting this erorr File "/Users/Sam/anaconda3/lib/python3.11/site-packages/sqlalchemy/orm/decl_base.py", line 1163, in _declarative_constructor
raise TypeError(
TypeError: 'year' is an invalid keyword argument for LapSummary

Your LapSummary model is missing year, location, session_name columns.

SQLAlchemy rejects `LapSummary(**lap_in.dict())` because schema has fields model doesn't.

Fix 1: Update Model (models/lap_summary.py)

```
from sqlalchemy import Column, Integer, String, Float
from db import Base
```

```
class LapSummary(Base):
    __tablename__ = "lap_summaries"

    id = Column(Integer, primary_key=True, index=True)
    session_key = Column(Integer, index=True, nullable=False)
    meeting_key = Column(Integer, index=True, nullable=True)  # Allow partial fills
    year = Column(Integer, index=True, nullable=True)         # ← ADD THESE
    location = Column(String, index=True, nullable=True)
    session_name = Column(String, index=True, nullable=True)

    driver_number = Column(Integer, index=True, nullable=False)
    lap_number = Column(Integer, nullable=False)
    lap_duration = Column(Float, nullable=True)
    max_speed_kph = Column(Float, nullable=True)
    avg_speed_kph = Column(Float, nullable=True)
    i1_speed = Column(Float, nullable=True)
    i2_speed = Column(Float, nullable=True)
    st_speed = Column(Float, nullable=True)
    label = Column(String, nullable=True)
```

Fix 2: Recreate DB Table

Option A: Delete & recreate (dev):

```
rm app.db  # Or your SQLite file
# Restart uvicorn → Base.metadata.create_all() runs
```

Option B: Alembic migration (prod):

```
alembic revision --autogenerate -m "add session metadata"
alembic upgrade head
```

Fix 3: Router Import Fix

Your router has **duplicate `create_lap_summary`**—remove the standalone def:

```
# DELETE this block:
# def create_lap_summary(...):  # ← Remove this

@router.post("", response_model=LapSummaryRead)  # Keep only this
def create_lap_summary_endpoint(  # Rename to avoid conflict
    lap_in: LapSummaryCreate,
```

```
    db: Session = Depends(get_db),
):
    return lap_service.create_lap_summary(db, lap_in)
```

Test Flow

1. **Add model columns** → recreate DB
2. **Remove duplicate function**
3. **Hit import**: `POST /import_from_openf1?year=2023&location=Belgium&session_name=Sprint Qualifying&driver_number=1`
   - Converter → `session_key=9140`
   - Creates rows with **all fields populated**

Now `lap_in.dict()` matches model exactly—no more `TypeError`.[63]

Quick check: Paste your current `LapSummary` model? Confirm missing fields.

***

@router.get("", response_model=List[LapSummaryRead])

def list_lap_summaries(
session_key: Optional[int] = Query(None),
year: Optional[int] = Query(None),
location: Optional[str] = Query(None),
session_name: Optional[str] = Query(None),
driver_number: Optional[int] = Query(None),
db: Session = Depends(get_db),
):
return lap_service.list_lap_summaries(
db, session_key, None, year, location, session_name, driver_number
) this function is broken ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: list_lap_summaries() takes from 1 to 3 positional arguments but 7 were given

`lap_service.list_lap_summaries` still has the **old signature** (only `db`, `session_key`, `driver_number`), so passing 7 args blows up.

You have two options; here's the clean one that matches your router.

1. Fix service signature

In `services/lap_summary.py`, change to:

```
from typing import List, Optional
from sqlalchemy.orm import Session
from models import LapSummary
from services.api_converter import get_openf1_session_keys  # if needed

def list_lap_summaries(
    db: Session,
    session_key: Optional[int] = None,
    meeting_key: Optional[int] = None,
    year: Optional[int] = None,
    location: Optional[str] = None,
    session_name: Optional[str] = None,
    driver_number: Optional[int] = None,
) -> List[LapSummary]:
    q = db.query(LapSummary)

    # If year+location+session_name provided, resolve to session_key
    if year is not None and location is not None and session_name is not None:
        keys = get_openf1_session_keys(year, location, session_name)
        session_key = keys["session_key"]
        meeting_key = keys.get("meeting_key")

    if session_key is not None:
        q = q.filter(LapSummary.session_key == session_key)
    if meeting_key is not None:
        q = q.filter(LapSummary.meeting_key == meeting_key)
    if driver_number is not None:
        q = q.filter(LapSummary.driver_number == driver_number)

    return q.all()
```

Now it matches your router call:

```
return lap_service.list_lap_summaries(
    db, session_key, None, year, location, session_name, driver_number
```

```
)
```

2. Or keep old service, adjust router

If you prefer not to change the service, simplify the router:

```
@router.get("", response_model=List[LapSummaryRead])
def list_lap_summaries(
    session_key: Optional[int] = Query(None),
    year: Optional[int] = Query(None),
    location: Optional[str] = Query(None),
    session_name: Optional[str] = Query(None),
    driver_number: Optional[int] = Query(None),
    db: Session = Depends(get_db),
):
    if year and location and session_name and not session_key:
        keys = get_openf1_session_keys(year, location, session_name)
        session_key = keys["session_key"]

    return lap_service.list_lap_summaries(
        db, session_key, driver_number
    )
```

This keeps `list_lap_summaries` in the service as:

```
def list_lap_summaries(
    db: Session,
    session_key: Optional[int] = None,
    driver_number: Optional[int] = None,
) -> List[LapSummary]:
    q = db.query(LapSummary)
    if session_key is not None:
        q = q.filter(LapSummary.session_key == session_key)
    if driver_number is not None:
        q = q.filter(LapSummary.driver_number == driver_number)
    return q.all()
```

Given your current code, **option 2 is quickest**: resolve `session_key` in the router, then keep the service simple.

Do you want the router to handle all conversion logic (simpler services) or centralize conversion inside the service layer?

⁂

create a git igoner for this project

Create a file named **.gitignore** in the project root with:

```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[cod]
*$py.class

# Virtual environments
.env
.env.*
venv/
.venv/
env/
.conda/
.pyenv/

# OS / Editor junk
.DS_Store
Thumbs.db
.vscode/
.idea/

# Logs
*.log
logs/

# SQLite / local DBs
*.db
*.sqlite3

# Test / coverage
htmlcov/
.tox/
.nox/
```

```
.coverage
.coverage.*
.cache
.pytest_cache/

# Build / dist
build/
dist/
*.egg-info/
.eggs/

# Jupyter
.ipynb_checkpoints/

# FastAPI / UVicorn reload artifacts
*.pid
```

This ignores Python artifacts, your virtual env, local SQLite DBs, and editor files, which is typical for a FastAPI + SQLAlchemy project.[74][75]

Do you keep your SQLite DB in the project root (e.g. `app.db`), or inside an `app/` folder?

***

1. https://docs.fastf1.dev/core.html
2. https://docs.fastf1.dev/fastf1.html
3. https://openf1.org
4. https://openf1.org/docs/
5. https://api-sports.io/documentation/formula-1/v1
6. https://owasp.org/www-community/vulnerabilities/Insufficient_Session-ID_Length
7. https://www.reddit.com/r/F1DataAnalysis/comments/1c91u26/any_public_f1_apis_which_include_sprint_results/
8. https://cran.r-project.org/web/packages/f1dataR/f1dataR.pdf
9. https://www.youtube.com/watch?v=HSV2FhPtRVI

10. https://docs.fastf1.dev/fastf1.html

11. https://docs.fastf1.dev/core.html

12. https://openf1.org

13. https://openf1.org/docs/

14. https://velog.io/@hjjwa1234/OpenF1-API-정리

15. https://github.com/theOehrly/Fast-F1/issues/625

16. https://docs.fastf1.dev/plotting.html

17. http://docs.fastf1.dev/core.html

18. https://openf1.org/auth.html

19. https://www.rdocumentation.org/packages/f1dataR/versions/2.0.1/topics/get_session

20. https://openf1.org

21. https://docs.fastf1.dev/core.html

22. https://openf1.org/docs/

23. https://openf1.org

24. https://lobehub.com/mcp/openf1-openf1-mcp-server

25. https://www.reddit.com/r/F1DataAnalysis/comments/16w84uz/openf1_an_api_for_realtime_f1_data/

26. https://github.com/ianleckey/openf1-js

27. https://mundobytes.com/en/What-is-the-openf1-api-for/

28. https://mcpservers.org/servers/AryaAkman/Open-F1-MCP-Server

29. https://www.linkedin.com/pulse/f1-project-querying-apis-using-python-arthur-ladwein-9lpec

30. https://www.f1matcha.com/telemetry/

31. https://github.com/elarswa/openf1-sdk

32. https://openf1.org/docs/

33. https://mundobytes.com/en/What-is-the-openf1-api-for/

34. https://openf1.org/auth.html

35. https://socket.dev/npm/package/openf1-client

36. https://www.reddit.com/r/F1DataAnalysis/comments/16w84uz/openf1_an_api_for_realtime_f1_data/

37. https://openf1.org

38. https://dlthub.com/workspace/source/formula-one

39. https://github.com/br-g/openf1/discussions/140

40. https://github.com/ianleckey/openf1-js

41. https://github.com/elarswa/openf1-sdk

42. https://openf1.org/docs/

43. https://www.perplexity.ai/search/7408be75-811f-49f9-8bae-8fce0e727e63

44. projects.f1_telemetry_api

45. https://openf1.org/docs/

46. https://openf1.org

47. https://github.com/br-g/openf1/discussions/148

48. https://github.com/bordanattila/OpenF1_tutorial

49. https://www.reddit.com/r/F1DataAnalysis/comments/16w84uz/openf1_an_api_for_realtime_f1_data/

50. https://mundobytes.com/en/What-is-the-openf1-api-for/

51. https://openf1.org/auth.html

52. https://lobehub.com/mcp/openf1-openf1-mcp-server

53. https://www.postman.com/maintenance-astronomer-29796265/f1-api/documentation/60v93se/formula-one-api

54. https://github.com/br-g/openf1

55. https://openf1.org/docs/

56. https://www.perplexity.ai/search/7408be75-811f-49f9-8bae-8fce0e727e63

57. https://openf1.org/docs/

58. https://openf1.org/docs/

59. https://www.perplexity.ai/search/d374e2a9-70b9-4256-b452-fcaa031c2924

60. https://openf1.org/docs/

61. https://openf1.org/docs/

62. https://openf1.org/docs/

63. https://www.perplexity.ai/search/d374e2a9-70b9-4256-b452-fcaa031c2924

64. https://stackoverflow.com/questions/75639004/positional-argument-error-when-trying-to-send-a-post-request

65. https://tutorialreference.com/python/examples/faq/python-error-typeerror-takes-2-positional-argument-but-3-were-given

66. https://github.com/tiangolo/typer/issues/173

67. https://github.com/encode/starlette/issues/608

68. https://www.reddit.com/r/learnpython/comments/l46r8j/how_to_deal_with_method_takes_1_positional/

69. https://stackoverflow.com/questions/70450606/typeerror-init-takes-1-positional-argument-but-2-were-given-in-fastapi

70. https://www.sololearn.com/en/Discuss/1722301/typeerror-**init**-takes-3-positional-arguments-but-7-were-given

71. https://github.com/fastapi/fastapi/discussions/7331

72. https://www.sololearn.com/en/Discuss/1722301/typeerror-**init**-takes-3-positional-arguments-but-7-were-given-

73. https://stackoverflow.com/questions/70909292/typeerror-evaluate-takes-3-positional-arguments-but-4-were-given-in-fastapi

74. https://github.com/mfreeborn/fastapi-sqlalchemy/blob/master/.gitignore

75. https://github.com/vahidrezazadeh/fastapi-example/blob/master/.gitignore

76. https://testdriven.io/courses/scalable-fastapi-aws/git/

77. https://github.com/grillazz/fastapi-sqlalchemy-asyncpg/blob/main/.gitignore

78. https://notes.kodekloud.com/docs/Python-API-Development-with-FastAPI/Deployment/Setup-Git

79. https://chaoticengineer.hashnode.dev/fastapi-sqlalchemy

80. https://fastapitutorial.com/blog/git-github-gitignore-for-fastapi/

81. https://testdriven.io/courses/serverless-fastapi/git/

82. https://pythonic-sqlalchemy-query.readthedocs.io/en/latest/.gitignore.html

83. https://dev.to/alexvanzyl/fastapi-simple-application-structure-from-scratch-2mem